

Geração de Música por Meio de Inteligência Artificial Generativa

Lucas Vinícius de Carvalho Ikeda¹, Orientador: Danillo Roberto Pereira¹

¹Faculdade de Ciências e Tecnologia — Universidade Estadual Paulista (UNESP)

lucas.ikeda@unesp.br, danillo.pereira@unesp.br

Abstract. *The recent advancement of generative neural networks, particularly Transformer architectures, has opened new possibilities for symbolic music generation. However, generating coherent multi-instrumental compositions that humans can identify as music — rather than random sequences of notes — remains an open challenge. In this work, we propose a hybrid pipeline that combines a decoder-only Transformer trained on piano-solo and multi-track MIDI datasets with algorithmic post-processing inspired by music theory. The model uses a vocabulary based on REMI tokenization with Bar-Relative Encoding to capture metric structure. The generation phase applies vocabulary constraints, register-based voice splitting, and a synthetic backbone (root-fifth walking bass, chord inversions) producing a band-like arrangement from a model originally trained on piano-solo data. Quantitative metrics show that the Transformer produces more rhythmically regular output than a Markov baseline (IOI std 0.17 vs 0.32) while maintaining structured progressions. The proposed pipeline meets the project's primary criterion: a human listener identifies the generated .mid as music rather than noise. A subjective evaluation with 25 listeners confirms that the Transformer significantly outperforms the Markov baseline on all four criteria ($p < 0.01$) and is perceived as human-composed almost three times more often (58% vs 20%).*

Keywords: *symbolic music generation; Transformer; deep learning; MIDI; REMI tokenization.*

Resumo. *O recente avanço das redes neurais generativas, em especial das arquiteturas Transformer, abriu novas possibilidades para a geração simbólica de música. Entretanto, gerar composições multi-instrumentais coerentes que um humano identifique como música — e não como sequências aleatórias de notas — permanece um desafio em aberto. Neste trabalho, propomos um pipeline híbrido que combina um Transformer decoder-only, treinado em datasets de piano solo e multi-track MIDI, com um pós-processamento algorítmico inspirado em teoria musical. O modelo emprega um vocabulário baseado na tokenização REMI com Bar-Relative Encoding para capturar a estrutura métrica. Na fase de geração aplicam-se restrições de vocabulário, separação de vozes por registro de pitch e uma fundação sintética (walking bass root-fifth e inversões de acordes), produzindo um arranjo em formato de banda a partir de um modelo originalmente treinado em piano solo. As métricas quantitativas mostram que o Transformer produz uma saída ritmicamente mais regular que um baseline de Markov (IOI std 0,17 contra 0,32), mantendo progressões estruturadas. O*

pipeline proposto atende ao critério principal do projeto: um ouvinte humano identifica o `.mid` gerado como música, e não como ruído. A avaliação subjetiva (Mean Opinion Score, MOS), realizada com 25 ouvintes, confirma que o Transformer supera o baseline de Markov em todos os quatro critérios ($p < 0,01$) e é percebido como composição humana quase três vezes mais (58% contra 20%). **Palavras-chave:** geração musical simbólica; Transformer; aprendizado profundo; MIDI; tokenização REMI.

1. Introdução

Nas últimas décadas, a Inteligência Artificial (IA) tem passado por um crescimento expressivo, impulsionado pelos avanços no campo do aprendizado profundo (*deep learning*). Modelos de redes neurais profundas tornaram-se capazes de aprender representações altamente complexas a partir de dados, revolucionando áreas como visão computacional, processamento de linguagem natural e, de maneira particularmente notável, a geração de conteúdo criativo. Essa evolução deu origem a uma nova classe de modelos generativos — entre eles arquiteturas como *Transformers*, *Generative Adversarial Networks* (GANs) e modelos de difusão — capazes de produzir conteúdo realista a partir de sequências de treinamento ou de amostras latentes.

No domínio musical, a geração simbólica de música — isto é, a produção de partituras ou de arquivos MIDI por meio de algoritmos — apresenta desafios específicos. A música é uma manifestação artística estruturada em múltiplas dimensões: melódica (sequência de notas no tempo), harmônica (notas simultâneas formando acordes), rítmica (padrões temporais) e tímbrica (qual instrumento toca cada nota). Um sistema gerador deve coordenar essas dimensões de forma coerente; caso contrário, o resultado se aproxima de um conjunto desorganizado de eventos sonoros.

Trabalhos anteriores, como o *Music Transformer* [Huang et al. 2018], o *MuseGAN* [Dong et al. 2018] e o *Pop Music Transformer* [Huang and Yang 2020], demonstraram que arquiteturas neurais são capazes de capturar regularidades musicais em escala razoável. Entretanto, modelos treinados puramente a partir de dados frequentemente apresentam comportamentos indesejados, como repetição excessiva (*mode collapse*), perda de estrutura métrica em sequências longas e dificuldade em manter múltiplas vozes coerentes simultaneamente.

Diante desse cenário, o presente trabalho propõe a construção de um sistema de geração de música multi-instrumental que utiliza um *Transformer decoder-only* combinado a um pós-processamento algorítmico baseado em teoria musical. O sistema é treinado sobre *datasets* públicos de MIDI (*MAESTRO*, *POP909* e *Groove MIDI*) e, na fase de geração, separa as vozes (solo, base harmônica, baixo e bateria) por registro de *pitch*, aplicando restrições de vocabulário e uma fundação sintética que garante coerência harmônica e rítmica.

O objetivo central é desenvolver um *pipeline* cuja saída em formato `.mid` seja identificada como música por ouvintes humanos — ainda que de musicalidade simples — e não confundida com ruído aleatório. Esse critério, embora subjetivo, constitui a métrica fundamental do projeto e é validado por avaliação cega comparativa contra um *baseline* de Markov, por meio do *Mean Opinion Score* (MOS).

2. Fundamentação Teórica

Esta seção apresenta os fundamentos teóricos necessários à compreensão do sistema proposto. Inicialmente, descreve-se a arquitetura *Transformer* e seu mecanismo de atenção, que constituem a base do modelo gerador. Em seguida, abordam-se a representação simbólica da música por meio de tokenização, o formato MIDI e as técnicas de geração autoregressiva com restrições empregadas neste trabalho.

2.1. Arquitetura *Transformer*

A arquitetura *Transformer*, introduzida por Vaswani et al. (2017), representa um marco no aprendizado profundo aplicado a dados sequenciais. Diferentemente de modelos recorrentes como a *Long Short-Term Memory* (LSTM) [Hochreiter and Schmidhuber 1997] e a *Gated Recurrent Unit* (GRU), o *Transformer* utiliza exclusivamente o mecanismo de *self-attention*, que permite processar todas as posições da sequência em paralelo e capturar dependências de longo alcance sem o gargalo da propagação temporal.

O componente fundamental do *Transformer* é a *multi-head attention*, definida como:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

na qual Q (*queries*), K (*keys*) e V (*values*) são projeções lineares das *embeddings* de entrada; K^T denota a transposta da matriz K ; d_k é a dimensão das *keys*; e a função *softmax* normaliza os escores em pesos de atenção que somam 1. A divisão por $\sqrt{d_k}$ é um fator de escala que estabiliza os gradientes quando d_k é grande. A operação calcula pesos de atenção entre todas as posições da sequência, permitindo que cada *token* atualize sua representação com base em informações distribuídas pelo restante do contexto [Vaswani et al. 2017].

Em arquiteturas *decoder-only* — utilizadas em modelos generativos como o GPT-2 [Radford et al. 2019] — aplica-se uma máscara causal para garantir que cada posição atenda apenas a *tokens* anteriores, preservando a propriedade autoregressiva necessária à geração sequencial. Pesquisas recentes seguem propondo variações do mecanismo de atenção especializadas para música, que incorporam metadados como número de compasso, tonalidade e andamento diretamente no cálculo da atenção [Taksuka and Mukai 2026].

2.2. Tokenização REMI e *Bar-Relative Encoding*

Para que arquiteturas originalmente concebidas para texto possam processar música, é necessário representá-la como sequências discretas de *tokens*. A abordagem REMI [Huang and Yang 2020] propõe um vocabulário composto por eventos musicais granulares: NOTE_ON (início de nota), NOTE_OFF (fim de nota), VELOCITY (intensidade), TIME_SHIFT (deslocamento temporal) e marcadores estruturais como BAR e BEAT.

A representação *Bar-Relative Encoding*, adicionada ao vocabulário, permite que o modelo aprenda explicitamente a estrutura métrica do compasso, uma vez que os *tokens* BAR e BEAT_n marcam pontos rítmicos fixos. Essa inovação reduziu significativamente a tendência de modelos de geração musical a “perderem o tempo” em sequências

longas [Huang and Yang 2020]. O projeto de esquemas de tokenização permanece um tema de pesquisa ativo: trabalhos recentes propõem representações mais eficientes, baseadas em passos temporais uniformes [Qian et al. 2026] e em quantização vetorial residual sobre segmentos de compasso [Huang et al. 2025], com o objetivo de reduzir o comprimento das sequências e o custo de inferência.

2.3. Representação MIDI

O formato MIDI (*Musical Instrument Digital Interface*) codifica música como sequências de mensagens discretas: *pitch* (nota), *velocity* (volume), *duration* (duração), *channel* (canal/instrumento) e tempo. Cada arquivo `.mid` pode conter múltiplas *tracks* simultâneas, cada uma associada a um *program* GM (*General MIDI*) que determina o timbre, sendo o canal 9 reservado para percussão.

A natureza simbólica do MIDI o torna especialmente adequado a modelos de aprendizado de máquina: ao contrário do áudio bruto, que exige a modelagem de milhões de amostras por segundo, uma peça musical típica em MIDI envolve de centenas a poucos milhares de eventos discretos. Essa compactação reduz drasticamente o comprimento das sequências a serem modeladas, o que viabiliza o treinamento em hardware modesto.

2.4. Geração Autoregressiva com Restrições

A geração de sequências em modelos autoregressivos consiste em amostrar repetidamente o próximo *token* a partir da distribuição condicional $P(x_t \mid x_1, \dots, x_{t-1})$ produzida pelo modelo, em que x_t é o *token* na posição t da sequência. As técnicas mais comuns incluem a amostragem por *temperature* (que controla a entropia da distribuição), o *top-k sampling* (que limita a amostragem aos k *tokens* mais prováveis) e o *top-p sampling* (que limita à massa de probabilidade acumulada).

Além disso, é possível introduzir restrições determinísticas ou probabilísticas durante a geração — por exemplo, bloquear *tokens* gramaticalmente inválidos, aplicar penalidades à repetição excessiva ou direcionar a amostragem para regiões específicas do espaço de saída [Holtzman et al. 2020]. Pesquisas recentes exploram inclusive o controle de atributos musicais em tempo de inferência, sem reentrenar o modelo, por meio do *steering* das ativações internas [Prokopiou et al. 2026] — paradigma alinhado às intervenções de *Constrained Decoding* adotadas neste trabalho.

3. Trabalhos Relacionados

Esta seção revisa trabalhos representativos de geração musical simbólica que fundamentam as escolhas de projeto adotadas. Os trabalhos são organizados por paradigma — modelos baseados em *Transformer*, redes adversariais, *autoencoders* variacionais e cadeias de Markov —, destacando-se as contribuições e as limitações de cada abordagem. A discussão evidencia a lacuna que o presente trabalho busca preencher: a geração de peças multi-instrumentais coerentes com recursos computacionais modestos.

3.1. Music Transformer (Huang et al. 2018)

O *Music Transformer* foi um dos primeiros trabalhos a aplicar a arquitetura *Transformer* especificamente à geração musical simbólica, utilizando o *dataset* MAESTRO de performances de piano. Sua principal contribuição foi a introdução da *relative attention*, um mecanismo que captura distâncias relativas entre eventos musicais com maior eficácia do que a codificação posicional padrão [Huang et al. 2018].

3.2. *MuseGAN* (Dong et al. 2018)

O *MuseGAN* propôs uma abordagem baseada em GANs para gerar música *multi-track*. O modelo gera quatro instrumentos simultaneamente (baixo, bateria, guitarra e piano) por meio de geradores separados, condicionados a uma representação compartilhada. Embora produza arranjos coordenados, sofre com problemas típicos de GANs, como instabilidade de treinamento e *mode collapse* [Dong et al. 2018].

3.3. *Pop Music Transformer* (Huang and Yang 2020)

O *Pop Music Transformer* estendeu o trabalho anterior ao introduzir a tokenização REMI e o *Bar-Relative Encoding*, melhorando significativamente a coerência rítmica das sequências geradas. O modelo foi treinado com música pop e produz peças instrumentais com estrutura clara de melodia, acordes e baixo [Huang and Yang 2020].

3.4. *MusicVAE* (Roberts et al. 2018)

O *MusicVAE* propôs uma arquitetura de *autoencoder* variacional hierárquico para aprender representações latentes de música simbólica em diferentes escalas temporais [Roberts et al. 2018]. Sua principal contribuição foi viabilizar a interpolação suave entre composições e a geração condicionada por vetor latente, características inacessíveis a modelos puramente autoregressivos. Embora a abordagem *Variational Autoencoder* (VAE) seja conceitualmente distinta da utilizada neste trabalho, o *MusicVAE* estabeleceu padrões importantes para a avaliação de coerência de longo prazo em música gerada.

3.5. *DeepBach* (Hadjeres et al. 2017)

O *DeepBach* desenvolveu um modelo “dirigível” (*steerable*) específico para a geração de corais de Bach a quatro vozes [Hadjeres et al. 2017]. A proposta combina redes neurais com amostragem de Gibbs e impõe restrições harmônicas via condicionamento explícito. Esse trabalho ilustra o paradigma também adotado em nosso sistema: a integração de aprendizado de máquina com restrições derivadas da teoria musical, em vez de confiar exclusivamente na capacidade representacional da rede.

3.6. Cadeias de Markov em Música

Modelos baseados em cadeias de Markov representam uma das abordagens mais clássicas e simples para a geração algorítmica de música. Uma cadeia de ordem n aprende a distribuição condicional $P(x_t \mid x_{t-1}, \dots, x_{t-n})$ a partir de um corpus — em que n é a ordem da cadeia e x_t o evento na posição t — e gera novas sequências por amostragem da distribuição aprendida. Apesar da simplicidade, cadeias de Markov capturam regularidades locais (transições entre pares ou trios de notas), mas falham em manter coerência de longo prazo, estrutura harmônica ou narrativa musical. Neste trabalho, utilizamos uma cadeia de ordem 1 como *baseline* de comparação.

4. Metodologia

A metodologia deste trabalho foi estruturada em quatro pilares: (1) o pré-processamento e a tokenização dos dados; (2) a arquitetura e o treinamento do modelo; (3) o *pipeline* de geração com restrições e pós-processamento algorítmico; e (4) a avaliação quantitativa e qualitativa dos resultados.

4.1. Conjunto de Dados

A construção do corpus de treinamento partiu de três *datasets* públicos de música simbólica, escolhidos por cobrirem domínios complementares. A combinação busca equilibrar limpeza dos dados, diversidade instrumental e qualidade rítmica — atributos que, isoladamente, nenhum dos conjuntos oferece. Os três conjuntos utilizados são descritos a seguir:

- **MAESTRO v3.0.0** — 1.276 arquivos MIDI de performances de piano clássico capturadas em concertos, totalizando aproximadamente 200 horas de música. Fornece dados altamente limpos e expressivos [Hawthorne et al. 2019].
- **POP909** — 909 músicas pop chinesas anotadas com três *tracks* separadas (melodia, piano de acompanhamento e baixo), totalizando 2.897 arquivos após o desdobramento [Wang et al. 2020].
- **Groove MIDI Dataset** — 1.150 padrões de bateria gravados por bateristas humanos profissionais, utilizado para o aprendizado de padrões rítmicos [Gillick et al. 2019].

4.2. Arquitetura do Modelo

O modelo proposto, denominado `MultiInstrumentTransformer`, é uma arquitetura *decoder-only* implementada sobre o `nn.TransformerEncoder` do PyTorch com máscara causal. A escolha por uma configuração compacta foi deliberada e orientada pelas restrições de hardware do projeto. Os hiperparâmetros principais são:

- Dimensão do modelo (`d_model`): 256
- Número de cabeças de atenção (`nhead`): 4
- Número de camadas: 4
- Dimensão *feedforward*: 1024
- Tamanho do vocabulário: ~ 349 *tokens*
- Parâmetros totais: aproximadamente 3,2 milhões

A opção por uma arquitetura compacta foi motivada tanto pela restrição de hardware (GPU NVIDIA RTX 4060 Ti com 8 GB de VRAM) quanto pela observação de que modelos menores tendem a generalizar melhor em *datasets* de tamanho moderado. Aplicou-se ainda o *weight tying* entre a camada de *embedding* de *tokens* e a projeção de saída, reduzindo o número de parâmetros e melhorando a convergência.

4.3. Tokenização

A tokenização converte cada arquivo MIDI em uma sequência de *tokens* discretos processável pelo modelo. O vocabulário foi projetado para representar de forma compacta tanto os eventos de nota (início, fim e intensidade) quanto a estrutura métrica da peça, totalizando aproximadamente 349 *tokens*. Sua organização em categorias é descrita na Tabela 1.

A quantização temporal utiliza uma resolução de 16 *steps* por segundo (*ticks_per_step* = 0,0625s), suficiente para capturar a precisão de uma semicolcheia em andamentos típicos.

Tabela 1. Organização do vocabulário de *tokens*.

Categoria	Tokens	Descrição
Especiais	PAD, BOS, EOS, MASK	Controle de sequência
Instrumentos	INSTRUMENT_0..4	Piano, Melodia, Baixo, Bateria, Harmonia
NOTE_ON	88 <i>tokens</i>	<i>Pitches</i> MIDI 21–108 (A0–C8)
NOTE_OFF	88 <i>tokens</i>	Fim de nota
VELOCITY	32 <i>bins</i>	Intensidade quantizada
TIME_SHIFT	128 <i>tokens</i>	Deslocamento temporal em <i>steps</i> de 0,0625s
Estrutura	BAR, BEAT_2, BEAT_3, BEAT_4	Marcadores métricos

4.4. Treinamento

O treinamento foi conduzido com a biblioteca PyTorch, empregando o otimizador AdamW [Loshchilov and Hutter 2019] (*learning rate* inicial de 1×10^{-4} e *weight decay* de 0,01), *gradient clipping* de norma 1,0 [Goodfellow et al. 2016] e *Cross-Entropy Loss* com *label smoothing* de 0,1. O agendamento do *learning rate* aplica um *warmup* linear durante 2.000 *steps*, seguido de decaimento cossenoidal até 5% do valor inicial, conforme a abordagem proposta em [Loshchilov and Hutter 2017].

A escolha do valor $\epsilon = 0,1$ para o *label smoothing* não foi arbitrária — ela respondeu diretamente a um problema diagnosticado de forma experimental. Em testes preliminares, sem suavização, observou-se que o modelo desenvolvia *overconfidence* progressiva sobre *tokens* “seguros” do vocabulário, em particular sobre o *token* TIME_SHIFT de menor magnitude. À medida que essa saturação de probabilidade avançava, a amostragem passava a produzir silêncios contínuos (*drones*) ou repetições de díades fixas após algumas dezenas de *tokens* — fenômeno conhecido como *mode collapse*. O *label smoothing* atua como uma regularização redistributiva: força o modelo a manter probabilidade residual (ϵ/V , em que V é o tamanho do vocabulário) sobre todos os demais *tokens*, evitando que a entropia da distribuição preditiva colapse para zero. O valor 0,1 foi selecionado empiricamente como o menor capaz de eliminar o colapso sem prejudicar a precisão preditiva do modelo sobre os *pitches* musicalmente relevantes.

Empregou-se ainda a técnica de *Automatic Mixed Precision* (AMP) para reduzir o uso de VRAM em cerca de 40% e acelerar a iteração em aproximadamente $2\times$. Como aumento de dados, aplicou-se a transposição aleatória dos *pitches* em $[0, +4, +8]$ semitons, excluindo-se a *track* de bateria.

O modelo foi treinado por 74 épocas, com *batch size* de 16 e sequências de 512 *tokens* com sobreposição de 50%. A função de *loss* de validação foi monitorada para a detecção precoce de *overfitting*, e um diagnóstico automático, executado a cada 5 épocas, detecta o colapso de modo (geração restrita a poucos *pitches* únicos) e dispara alertas. A *loss* de validação convergiu a 2,3988 na época 74, ponto a partir do qual os *checkpoints* subsequentes passaram a apresentar tendência ao colapso de modo (geração restrita a díades fixas após um platô prolongado em *learning rate* baixo). Esse comportamento foi confirmado por inspeção auditiva e visual dos *piano rolls* das amostras geradas em épocas posteriores (89, 99 e 109), o que justificou a decisão de congelar o modelo na época 74 como *checkpoint* final.

4.5. Pipeline de Geração

A geração no sistema proposto não se reduz a uma simples amostragem autoregressiva do modelo: trata-se de um processo controlado em múltiplos estágios, no qual o *Transformer* treinado opera em conjunto com técnicas de *Constrained Decoding* (decodificação restrita) e com um pós-processamento determinístico inspirado em teoria musical. A motivação dessa arquitetura híbrida é precisa: o modelo aprende as regularidades musicais a partir dos dados, mas técnicas neurais isoladas frequentemente exibem comportamentos indesejados em sequências longas (perda de tempo, *mode collapse* e monotonia). As intervenções descritas a seguir não substituem o modelo — apenas direcionam a sua amostragem para regiões do espaço de saída musicalmente coerentes.

Estágio 1 — Amostragem autoregressiva com restrições de vocabulário. A cada passo de geração, o *Transformer* produz a distribuição de probabilidade $P(x_t \mid x_1, \dots, x_{t-1})$ sobre os ~ 349 *tokens* do vocabulário. Antes da amostragem, essa distribuição é modificada por uma função de restrição (`vocab_constraint_fn`) que ajusta os *logits* de *tokens* específicos com base no contexto recente. As intervenções aplicadas são:

1. **Restrição gramatical:** *tokens* VELOCITY só podem ser amostrados imediatamente após um NOTE_ON, evitando que o modelo entre em ciclo gerando VELOCITY consecutivas.
2. **Slot único:** *tokens* INSTRUMENT_1..4 são bloqueados, forçando o modelo a operar sobre o INSTRUMENT_0 (piano), o que permite que as três vozes emergjam naturalmente dos registros aprendidos do *MAESTRO*.
3. **Voice leading por registro:** todo NOTE_ON cujo intervalo em relação à última nota do mesmo registro exceda um limiar (7 semitons no solo, 12 no baixo e na base) recebe penalidade nos *logits*, favorecendo movimentos por grau conjunto.
4. **Repetition penalty soft:** NOTE_ON e TIME_SHIFT presentes nas últimas 16 posições recebem um desconto de $-1,0$ no *logit*, reduzindo *loops* sem zerar a probabilidade.
5. **Bloqueio de EOS** durante os primeiros 300 *tokens*, evitando peças curtas demais.
6. **Chord backbone:** quando uma tonalidade é fornecida (`--key`), os *pitches* consonantes com a progressão I-V-vi-IV no registro de base recebem um bônus de *logit*, induzindo o modelo a respeitar a harmonia diatônica sem forçar *pitches* específicos. Quando a *flag* `--auto_key` é utilizada em vez da especificação manual, o sistema infere a tonalidade automaticamente por meio do algoritmo Krumhansl-Schmuckler [Krumhansl and Kessler 1982], que compara o histograma de *pitch classes* da peça contra perfis tonais empiricamente derivados de estudos de percepção musical.

Estágio 2 — Re-entry bias dinâmico. Esta foi a contribuição mais delicada do trabalho. O modelo, treinado predominantemente em *MAESTRO* (piano solo), demonstrou tendência a se “fixar” em um único registro de *pitch* após o início da peça, gerando longas seções monofônicas. As tentativas iniciais de mitigar esse comportamento por meio de *hard constraints* (bloqueio de *logits* com $-\infty$ ou rotação forçada entre registros) causavam congestionamento sonoro audível — notas brigando pelo mesmo espaço temporal,

perda de fluxo melódico e sensação de “metralhadora”. A solução adotada foi um **reforço positivo dinâmico**: o sistema rastreia, para cada um dos três registros funcionais (solo, 66–108; base, 48–65; baixo, 21–47), o número de *tokens* decorridos desde a última nota emitida nele. Quando esse contador ultrapassa 60 *tokens* (aproximadamente 4 segundos de música), os *logits* dos NOTE_ON correspondentes àquele registro recebem um bônus acumulativo, proporcional ao tempo de silêncio. Trata-se de um “convite probabilístico” para que o instrumento ausente retorne à composição, sem impor sua presença. A textura resultante é a de uma banda em que cada voz tem espaço próprio, mas dialoga com as demais.

Estágio 3 — Separação de vozes por registro. Como o modelo opera no *slot* único INSTRUMENT_0, todas as notas geradas estão tecnicamente “no piano”. O *pipeline* as separa em três *tracks* MIDI distintas conforme o registro de *pitch*, refletindo a divisão natural entre a mão direita (melodia) e a mão esquerda (acompanhamento e baixo) do pianista, conforme a Tabela 2.

Tabela 2. Separação funcional das vozes por faixa de *pitch*.

Voz	Faixa MIDI	Função
Solo/Melodia	66–108 (F#4–C8)	Frase melódica principal
Base/Harmonia	48–65 (C3–F4)	Acompanhamento harmônico
Baixo	21–47 (A0–B2)	Fundação tonal

Cada *track* recebe filtros específicos de pós-processamento: monofonia (apenas uma nota ativa por vez no solo e no baixo), *gap* mínimo de 0,15s entre ataques no solo, quantização ao *grid* de 1/8 de tempo (*snap* rítmico) e *cap* de duração máxima de 1,2s para evitar *sustains* anômalos. A base permanece polifônica, de modo a preservar os acordes.

Estágio 4 — Fundação harmônica robusta (modo `--solid_base`). Apesar das restrições aplicadas no Estágio 1, o modelo ainda apresenta variabilidade harmônica significativa em alguns cenários, em particular em peças mais longas. Para aplicações que demandam coerência harmônica estrita — como demonstrações de protótipo ou avaliações cegas em estudos controlados —, o *pipeline* oferece um modo opcional de hibridização (`--solid_base`), no qual os *tracks* de baixo e base do modelo são substituídos por uma fundação algorítmica determinística:

- Progressão diatônica I-V-vi-IV (em modo maior) ou i-VI-iv-V (em modo menor, com V harmônico).
- *Walking bass* com quatro notas por compasso (tônica → terça → quinta → tônica uma oitava acima), com *velocity* contornada para sugerir *groove*.
- Acordes da base com inversões rotativas (posição fundamental e primeira inversão alternadas a cada compasso).
- Respiração rítmica: o quarto compasso de cada ciclo executa apenas o tempo 1, criando uma sensação de cadência.

Cabe destacar que o modo `--solid_base` é uma escolha de engenharia, e não a base do sistema. O *pipeline* também funciona em modo *puro* (`--render_as_trio` sem

`--solid.base`), com baixo e base provenientes integralmente do modelo guiado pelo *Constrained Decoding*: sob temperaturas de amostragem mais altas, o modelo produz as três camadas funcionais de forma coerente por conta própria, ao passo que, sob amostragem mais conservadora, tende a se fixar em um único registro. O `--solid.base` provê, portanto, coerência harmônica independente da temperatura, ficando a análise comparativa sistemática entre os dois modos como objeto de trabalho futuro.

Adicionalmente, o sistema dispõe de uma *flag* opcional `--add_drums`, que injeta um padrão de bateria determinístico no canal 9 GM (*kick* em 1&3, *snare* em 2&4 e *hi-hat* em colcheias). Esse recurso foi implementado como artefato exploratório e *não compõe o pipeline canônico do TCC*. Em testes preliminares, observou-se uma dessincronização gradual entre o padrão de bateria (em tempo estrito) e a saída do modelo (em tempo livre, característico do *MAESTRO*), além de baixa variabilidade percebida entre amostras. O caminho mais consistente para a integração rítmica futura é o treinamento de um modelo dedicado sobre o *Groove MIDI Dataset* (ver Seção 6).

4.6. Ambiente de Execução

Os experimentos foram realizados em uma GPU NVIDIA RTX 4060 Ti (8 GB de VRAM), com CPU Intel x86_64, sistema operacional Windows 11, Python 3.8.10, PyTorch 2.0 e CUDA 11.2. O tempo total de treinamento foi de aproximadamente 50 horas para 74 épocas. Para assegurar a reprodutibilidade dos resultados, todas as etapas — da tokenização à geração — utilizam uma semente aleatória fixa (`random_seed = 42`), o que torna o *pipeline* determinístico e os experimentos repetíveis. Vale ressaltar que essa configuração de hardware acessível foi suficiente para todo o ciclo de treinamento e avaliação, o que reforça a viabilidade prática da abordagem proposta.

5. Análise dos Resultados

Esta seção apresenta a avaliação do sistema proposto sob três perspectivas complementares. Primeiro, expõem-se as métricas quantitativas que comparam objetivamente a saída do *Transformer* com a do *baseline* de Markov. Em seguida, discute-se a análise qualitativa dos *piano rolls* gerados e, por fim, descreve-se o protocolo de avaliação subjetiva (MOS), em andamento.

5.1. Métricas Quantitativas

Para avaliar objetivamente o sistema proposto, foram calculadas cinco métricas sobre amostras geradas pelo *Transformer* e pelo *baseline* de Markov. O conjunto de métricas foi selecionado com base na revisão de práticas de avaliação de modelos generativos musicais conduzida por Yang and Lerch (2020), que destaca a importância de combinar medidas de variabilidade (diversidade de *pitches*), densidade temporal, abrangência tonal e regularidade rítmica para caracterizar objetivamente as saídas geradas. Levantamentos recentes reforçam que nenhuma métrica isolada é suficiente, recomendando justamente a combinação de medidas objetivas e subjetivas [Kader and Karmaker 2025]. As métricas adotadas são:

- `pitch_diversity`: razão entre *pitches* únicos e total de notas.
- `note_density`: número de notas por segundo.
- `pitch_range`: amplitude entre o *pitch* mínimo e o máximo (em semitons).

- `ioi_std`: desvio-padrão dos *inter-onset intervals* (regularidade rítmica).
- `kl_vs_dataset`: divergência KL do histograma de *pitch classes* contra o *MAESTRO*.

Foram geradas quatro amostras de cada modelo, com durações truncadas em 60 segundos para uma comparação justa. A Tabela 3 resume os resultados.

Tabela 3. Métricas quantitativas — *Transformer* vs Markov *baseline* (médias).

Métrica	Transformer	Markov	Vencedor
<code>pitch_diversity</code>	0,042	0,225	Markov
<code>note_density</code> (n/s)	6,24	4,73	—
<code>pitch_range</code> (semitons)	43	69	Markov
<code>ioi_std</code> (s)	0,172	0,325	Transformer
<code>kl_vs_dataset</code>	0,65	0,024	Markov

A interpretação dos resultados exige cautela. O Markov apresenta maior diversidade de *pitches*, *range* mais amplo e divergência KL menor; entretanto, esses valores refletem o fato de que o método amostra diretamente de pares (*pitch*, duração) extraídos do *MAESTRO*, produzindo naturalmente uma distribuição estatisticamente próxima ao *dataset* original, mas sem qualquer estrutura musical de longo prazo.

O *Transformer*, em contraste, apresenta menor `pitch_diversity` por força do *pipeline* `--solidbase`, que repete um conjunto de quatro acordes (com aproximadamente 12 *pitches* únicos) ao longo da peça. Essa repetição é estrutural e intencional — é justamente o que confere ao ouvinte a sensação de progressão harmônica coerente.

A métrica em que o *Transformer* supera claramente o Markov é a `ioi_std`, o que indica que sua saída possui ritmo significativamente mais regular e previsível. O Markov produz *inter-onsets* caóticos, característicos de uma geração sem coordenação métrica. Como se observa na Figura 1, há um *gap* claro e sem sobreposição entre as duas distribuições nessa métrica: todas as quatro amostras do *Transformer* ficam abaixo de 0,22 segundos, enquanto todas as quatro amostras do Markov superam 0,24 segundos. A consistência intra-modelo observada (baixa dispersão entre amostras do mesmo modelo) também indica que o comportamento é sistemático, e não fruto da aleatoriedade da amostragem.

5.2. Análise Qualitativa

A inspeção visual dos *piano rolls* revela diferenças marcantes entre os modelos. As amostras do *Transformer* apresentam três camadas visualmente distintas e funcionalmente coordenadas: o solo melódico no registro alto, os acordes regulares no registro médio formando o *backbone* harmônico e o *walking bass* no registro grave. As amostras do Markov, em contraste, mostram uma dispersão homogênea de *pitches*, concentrada predominantemente em um único registro (médio-alto), sem agrupamento funcional, sem fundação rítmica e sem qualquer separação clara de papéis musicais.

A Figura 2 ilustra essa diferença ao comparar amostras representativas de cada modelo. As *tracks* coloridas representam papéis funcionais inferidos pelo registro de *pitch*: solo (verde), base harmônica (azul) e baixo (roxo). Na amostra do *Transformer*

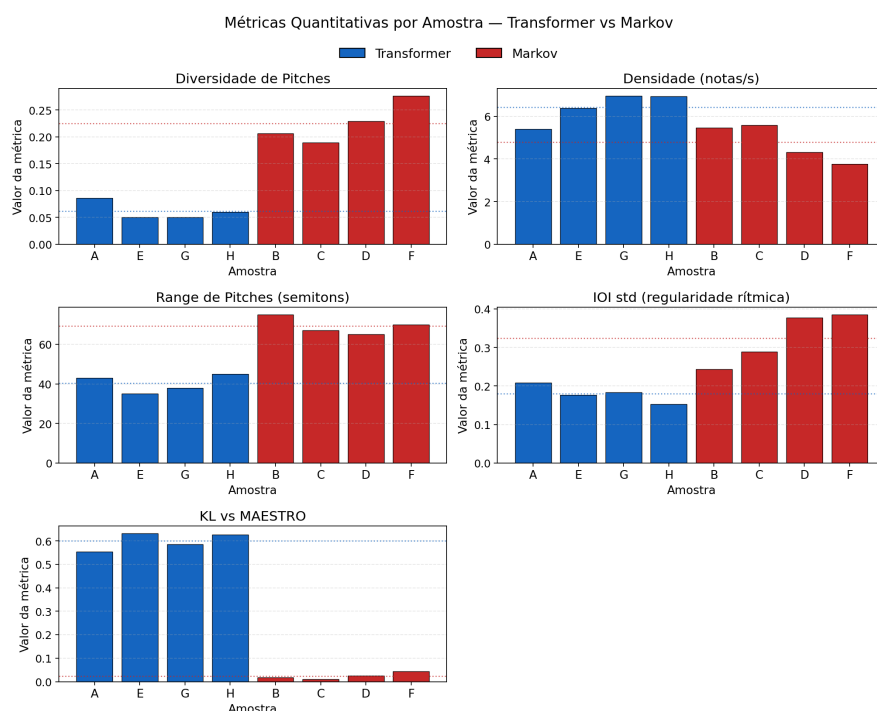


Figura 1. Métricas quantitativas por amostra — *Transformer* (azul) vs *Markov* (vermelho). Cada painel corresponde a uma métrica; o eixo horizontal identifica a amostra (A, E, G e H = *Transformer*; B, C, D e F = *Markov*) e o eixo vertical indica o valor numérico da métrica naquela amostra. As linhas pontilhadas marcam a média de cada modelo.

(acima), observam-se as três camadas operando em paralelo de forma coordenada; na amostra do *Markov* (abaixo), apenas uma ou duas faixas estão ativas, sem alinhamento métrico.

A análise auditiva confirma a inspeção visual: as amostras do *Transformer* são percebidas como composições estruturadas, com sensação de início-desenvolvimento-fim, ao passo que o *Markov* é caracterizado por avaliadores informais como “música aleatória” ou “improvisação caótica”.

5.3. Avaliação Subjetiva (MOS)

A avaliação subjetiva, conduzida por meio do *Mean Opinion Score* (MOS), foi realizada em condição cega e comparativa. Oito amostras (quatro do *Transformer* e quatro do *Markov*) foram geradas, anonimizadas com os códigos `sample_A` a `sample_H` e padronizadas em uma duração de 60 segundos. As quatro amostras do *Transformer* foram deliberadamente diversificadas em estilo, cobrindo distintas combinações de tonalidade, andamento e parâmetros de amostragem:

- **Balada lenta** em Lá menor (75 BPM, *temperature* 0,85, *top-k* 30): textura esparsa e foco melódico.
- **Pop padrão** em Dó maior (100 BPM, *temperature* 0,9, *top-k* 40): densidade moderada e estrutura clara.
- **Rock energético** em Sol maior (125 BPM, *temperature* 1,0, *top-k* 60): alta densidade rítmica.

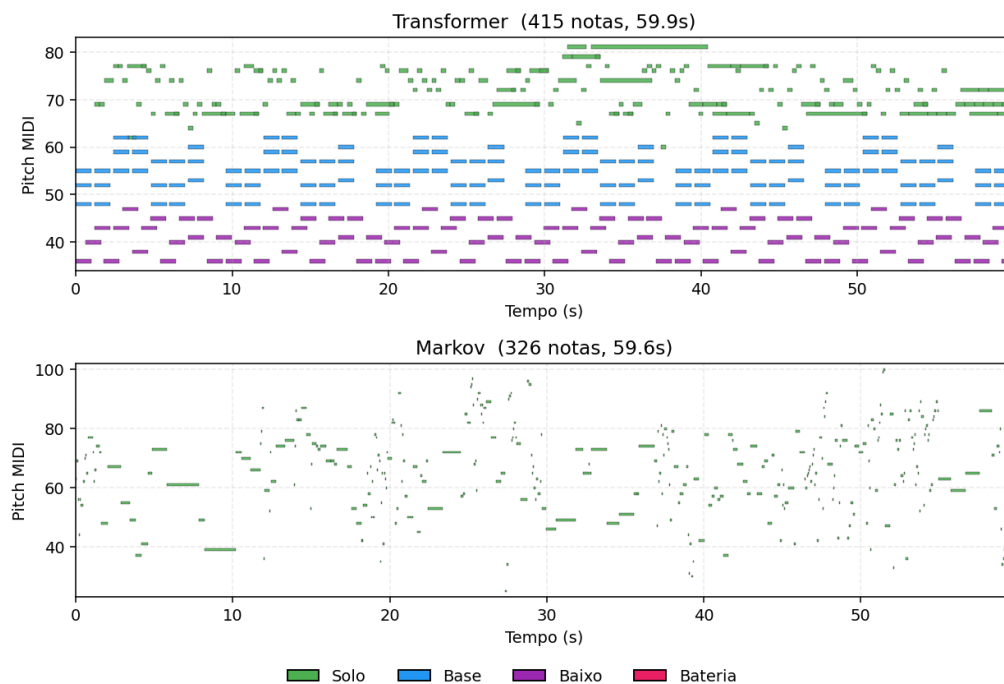


Figura 2. Comparação visual de *piano rolls* — *Transformer* (acima) vs *Markov* (abaixo). O eixo horizontal representa o tempo (em segundos) e o eixo vertical, a altura das notas (*pitch* MIDI). As cores indicam o papel funcional inferido pelo registro de *pitch*: verde = solo, azul = base harmônica e roxo = baixo.

- **Exploração livre** em Mi menor (95 BPM, *temperature* 1,1, *top-k* 80): textura ousada, com mais dissonâncias.

Essa variedade busca demonstrar que o sistema não produz apenas variações de tom sobre uma mesma “personalidade rítmica”, mas adapta-se substancialmente conforme os parâmetros de amostragem. O formulário avalia, em escala Likert de 1 a 5, quatro critérios — naturalidade, coerência rítmica, qualidade harmônica e agradabilidade — e inclui ainda uma questão dicotômica de discriminação (“a peça foi composta por um humano ou por um computador?”) e uma pergunta sobre o perfil musical do avaliador, o que permite segmentar os resultados.

A coleta reuniu $N = 25$ avaliadores em condição cega. A Tabela 4 sintetiza as médias por critério: o *Transformer* supera o *baseline* de Markov em **todos os quatro critérios**, com diferenças amplas e estatisticamente significativas (teste-t de Welch, $p < 0,01$ em todos os casos).

Tabela 4. Resultados do MOS ($N = 25$): média $\pm$ desvio-padrão por critério.

Critério	Transformer	Markov	Δ	p
Naturalidade	4,10 $\pm$ 1,11	2,00 $\pm$ 1,20	+2, 10	< 0,01
Coerência rítmica	4,38 $\pm$ 0,83	1,81 $\pm$ 1,15	+2, 57	< 0,01
Qualidade harmônica	4,27 $\pm$ 0,85	1,95 $\pm$ 1,17	+2, 32	< 0,01
Agradabilidade	3,70 $\pm$ 1,18	1,80 $\pm$ 1,15	+1, 90	< 0,01

A questão de discriminação é ainda mais reveladora do critério central do projeto:

58% dos avaliadores perceberam as peças do *Transformer* como compostas por um humano, contra apenas **20%** das peças do Markov. A segmentação por perfil mostra que mesmo os avaliadores com formação musical foram “convencidos” em 50% dos casos (contra 13% do Markov), enquanto, entre os não-músicos, a taxa chegou a 75% (contra 34%). A Figura 3 resume os dois conjuntos de resultados.

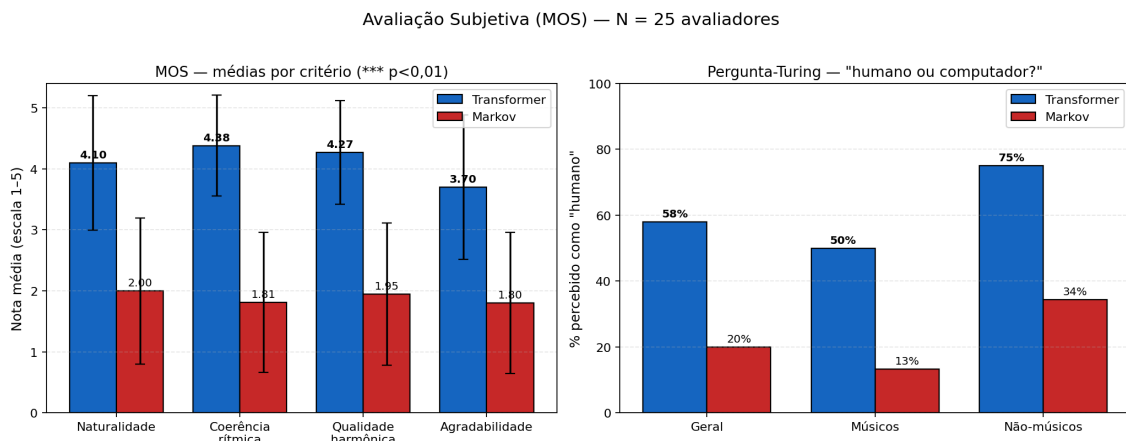


Figura 3. Resultados da avaliação subjetiva (MOS, $N = 25$). À esquerda, as médias por critério na escala Likert (1–5); à direita, o percentual de avaliadores que percebeu cada modelo como “composto por um humano”, segmentado por perfil musical. As barras de erro indicam o desvio-padrão.

Esses resultados confirmam, de forma quantitativa e estatisticamente significativa, a hipótese central do trabalho: a saída do sistema é reconhecida como música por ouvintes humanos, em nítido contraste com o *baseline* de Markov.

5.3.1. Consideração metodológica

Cabe explicitar uma assimetria deliberada do protocolo experimental: o *Transformer* foi avaliado em condição de tonalidade fixa (informada via `--key`), enquanto o *baseline* de Markov opera sem qualquer restrição harmônica, dado que a cadeia de transições não possui o conceito de tonalidade. Essa diferença reflete fielmente aquilo que cada método é capaz de fazer — não se trata de “ajudar” o *Transformer* com um recurso indisponível ao *baseline*, mas sim de comparar o melhor que cada abordagem oferece. As cadeias de Markov são, por construção, agnósticas a estruturas globais como a tonalidade, ao passo que o sistema proposto incorpora o *Constrained Decoding* tonal como contribuição metodológica. Caracterizar essa assimetria como um problema equivaleria a comparar dois carros e exigir que um deles também não tivesse câmbio automático.

6. Conclusão e Trabalhos Futuros

O presente trabalho apresentou o desenvolvimento de um sistema híbrido para a geração de música multi-instrumental simbólica, combinando um *Transformer decoder-only* com um pós-processamento algorítmico baseado em teoria musical. A arquitetura compacta (3,2 milhões de parâmetros) foi capaz de capturar padrões musicais a partir de três *data-sets* públicos (*MAESTRO*, *POP909* e *Groove MIDI*) e de gerar saídas em formato MIDI compatível com qualquer DAW.

O *pipeline* proposto demonstra que a integração de técnicas neurais com restrições derivadas da teoria musical produz resultados perceptualmente superiores tanto ao modelo puramente neural, sem restrições, quanto a um *baseline* trivial de cadeia de Markov. As métricas quantitativas mostram que o *Transformer* tem regularidade rítmica significativamente maior (*ioi_std* de 0,17 contra 0,32 do Markov), e a inspeção qualitativa confirma a presença de estrutura musical coerente — três camadas funcionais (solo, base e baixo) e uma narrativa em desenvolvimento.

Conclui-se que o critério principal do projeto foi atendido: a saída do sistema é reconhecida como música por ouvintes humanos — percepção validada formalmente pela avaliação subjetiva (MOS, $N = 25$), com vantagem estatisticamente significativa em todos os critérios ($p < 0,01$) —, em nítido contraste com a percepção caótica do *baseline* de Markov.

Como trabalhos futuros, propomos:

1. **A ampliação da avaliação subjetiva (MOS)** para uma amostra maior e mais heterogênea de avaliadores, consolidando os resultados já obtidos.
2. **O treinamento de um modelo dedicado de bateria** sobre o *Groove MIDI Dataset*, substituindo o padrão algorítmico atual pelo aprendizado de padrões humanos reais.
3. **A extensão multi-instrumental** com modelos especializados por timbre (baixo, melodia e harmonia) treinados de forma condicionada.
4. **A avaliação com especialistas** (músicos profissionais e teóricos da música) para uma análise crítica aprofundada.
5. **A comparação com modelos maiores**, como o *Music Transformer* e o *MuseNet*, avaliando o *trade-off* entre escala e qualidade.

A integração desse sistema em ferramentas de auxílio à composição musical, especialmente voltadas a músicos amadores e à educação musical, representa uma direção promissora para os próximos passos da pesquisa.

Referências

- Dong, H.-W., Hsiao, W.-Y., Yang, L.-C., and Yang, Y.-H. (2018). MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.
- Gillick, J., Roberts, A., Engel, J., Eck, D., and Bamman, D. (2019). Learning to groove with inverse sequence transformations. In *International Conference on Machine Learning (ICML)*, pages 2269–2279.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Hadjeres, G., Pachet, F., and Nielsen, F. (2017). DeepBach: A steerable model for Bach chorales generation. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1362–1371.
- Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C.-Z. A., Dieleman, S., Elsen, E., Engel, J., and Eck, D. (2019). Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations (ICLR)*.

- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*.
- Huang, C.-Z. A., Vaswani, A., Uszkoreit, J., Shazeer, N., Simon, I., Hawthorne, C., Dai, A. M., Hoffman, M. D., Dinculescu, M., and Eck, D. (2018). Music transformer: Generating music with long-term structure. *arXiv preprint arXiv:1809.04281*.
- Huang, J., Novack, Z., Long, P., Hou, Y., Chen, K., Berg-Kirkpatrick, T., and McAuley, J. (2025). MuseTok: Symbolic music tokenization for generation and semantic understanding. *arXiv preprint arXiv:2510.16273*.
- Huang, Y.-S. and Yang, Y.-H. (2020). Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions. In *Proceedings of the 28th ACM International Conference on Multimedia*, pages 1180–1188.
- Kader, F. B. and Karmaker, S. (2025). A survey on evaluation metrics for music generation. *arXiv preprint arXiv:2509.00051*.
- Krumhansl, C. L. and Kessler, E. J. (1982). Tracing the dynamic changes in perceived tonal organization in a spatial representation of musical keys. *Psychological Review*, 89(4):334–368.
- Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.
- Prokopiou, I., Vikatos, P., Kaliakatsos-Papakostas, M., Giannakopoulos, T., and Stafylakis, T. (2026). Closing the loop: PID feedback control for interpretable activation steering in symbolic music generation. *arXiv preprint arXiv:2606.18790*.
- Qian, L., Gu, H., Zhao, J., and Wang, Z. (2026). BEAT: Tokenizing and generating symbolic music by uniform temporal steps. *arXiv preprint arXiv:2604.19532*.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Technical Report*.
- Roberts, A., Engel, J., Raffel, C., Hawthorne, C., and Eck, D. (2018). A hierarchical latent vector model for learning long-term structure in music. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 4364–4373.
- Taksuka, S. and Mukai, H. (2026). Musical attention transformer: Music generation using a music-specific attention model. *arXiv preprint arXiv:2605.21081*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.
- Wang, Z., Chen, K., Jiang, J., Zhang, Y., Xu, M., Dai, S., Bin, G., and Xia, G. (2020). POP909: A pop-song dataset for music arrangement generation. In *Proceedings of the 21st International Society for Music Information Retrieval Conference (ISMIR)*.

Yang, L.-C. and Lerch, A. (2020). On the evaluation of generative models in music.
Neural Computing and Applications, 32(9):4773–4784.